A MEAN Stack Proof of Concept for Student Admission Management: Design and Implementation of the IEM Admission Portal

Debarghaya Mitra Arka Ghosh Dastider Jitesh Jana Jatin Kumar Diptomoy Das

Department of Computer Science & Engineering
PCCCS594 – Software Engineering and Project Management Laboratory, Semester V
Team 404 Brain Not Found, Group 10
Institute of Engineering and Management, Kolkata, India

***Abstract*—Managing student admissions means collecting applications, supporting documents and decisions from many applicants and keeping each record visible only to the right people. This report presents a Proof of Concept (POC) of the IEM Admission Portal, a web application built on the MEAN stack (MongoDB, Express.js, Angular and Node.js). Applicants can register, submit an admission application, upload documents and track their application status; administrators can review applications, filter records, update statuses and view department-wise and status-wise statistics. Validation and access control are enforced on the backend rather than only in the interface: passwords are hashed with bcrypt, protected routes use JWT authentication, applicant and administrator operations are separated by role, applicants can reach only their own records, and uploads are limited to PDF, JPG or PNG files of at most 5 MB. We describe the design choices, the request-response cycle, the implemented features and the limitations plainly: payment is a demonstration status only, there is no email or OTP verification, and full testing and production deployment remain future work.**

***Index Terms*—Admission Portal, MEAN Stack, Proof of Concept, MongoDB, Express.js, Angular, Node.js, JWT Authentication, Role-Based Access Control, Software Engineering.**

I. INTRODUCTION

The IEM Admission Portal is a web-based application developed as a Proof of Concept using the MEAN stack. Its purpose is to simplify the student admission process by letting applicants register, submit admission applications, upload documents and track the status of their application, with a demonstration payment-status feature standing in for fee payment. It also provides an administrative interface through which authorised administrators review applications, verify documents and update admission statuses.

The POC demonstrates the complete request-response cycle between the Angular frontend, the Express and Node.js backend, and the MongoDB database. It concentrates on the core admission workflow identified in the project's Software Requirements Specification (SRS) [1].

This report documents that implementation. Our contributions are: (i) a working MEAN stack implementation of the core applicant and administrator workflows; (ii) a separation-of-concerns design in which Angular handles presentation, Express handles logic, validation and authorisation, and MongoDB handles persistence; (iii) backend-enforced validation and access control; (iv) a deployed application and public source repository; and (v) an explicit account of the limitations that separate this POC from a production admission system.

II. REQUIREMENTS AND SCOPE

A. Core Workflow

The POC follows the admission workflow documented in the SRS [1]. An applicant creates an account, submits an application, uploads supporting documents and follows the application until a decision is recorded. An administrator views the submitted applications, filters them, verifies documents and moves each application through the review stages to a final selection or rejection.

B. Out of Scope

Merit calculation, counselling and seat allocation are not included because they lie outside the currently documented project requirements. The POC is a foundation for the admission process, not a complete production-ready system.

III. TECHNOLOGY STACK AND DESIGN CHOICES

A. Technologies

Table I lists the technologies selected for the implementation and the purpose each serves.

TABLE I
TECHNOLOGY STACK

Technology	Purpose
MongoDB	Stores applicant details, application records and related information.
Express.js	Handles API routes, requests, validation and server-side operations.
Angular	Provides the applicant and administrator user interfaces.
Node.js	Runs the backend application and handles server-side execution.
Mongoose	Defines database schemas and communicates with MongoDB.
JWT	Supports authenticated access to protected routes [2].
bcrypt	Hashes passwords before they are stored in the database [3].
Multer	Handles document uploads.

B. Why the MEAN Stack

The MEAN stack was chosen because it uses JavaScript and TypeScript across all application layers and supports a modular client-server architecture. MongoDB provides flexible document storage, Express and Node.js handle backend

operations, and Angular supports a structured, component-based interface.

C. Separation of Concerns

Angular is responsible for displaying information and interacting with users. Express handles business logic, validation and authorisation. MongoDB is used only for persistent storage. Keeping these responsibilities apart makes the application easier to maintain and extend.

IV. SYSTEM ARCHITECTURE AND IMPLEMENTATION

A. Request-Response Cycle

The application follows a standard request-response cycle, summarised in Fig. 1. (1) The applicant performs an action on the Angular interface. (2) Angular sends an HTTP request to the Express API. (3) Express receives the request and executes the required backend logic. (4) Mongoose communicates with MongoDB to retrieve or update the required records. (5) MongoDB returns the requested data to the backend. (6) Express sends a JSON response to Angular. (7) Angular updates the interface based on the response.

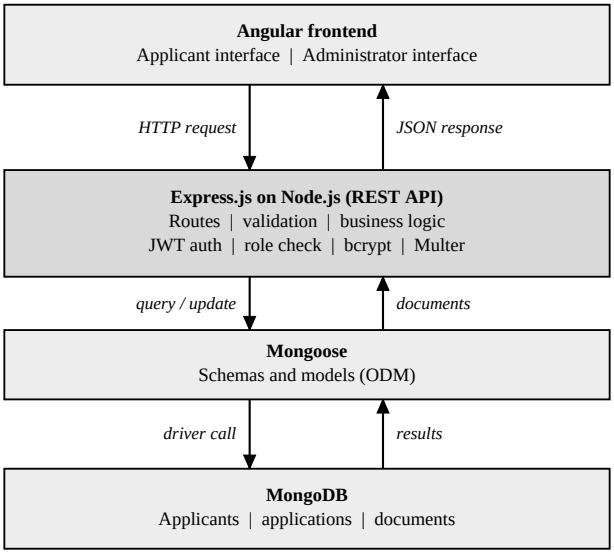

Fig. 1. Layered architecture and request-response path of the IEM Admission Portal. Requests travel down through Express and Mongoose to MongoDB; JSON responses travel back up to Angular.

B. Implemented Features

Table II summarises the functionality implemented in the POC.

TABLE II
IMPLEMENTED FEATURES

#	Feature	Description
1	Registration and login	Applicants create accounts and log in with their credentials.
2	Admission application	Registered applicants submit their admission details through the application form.
3	Document upload	Supporting documents can be uploaded in PDF, JPG or PNG format.
4	Status tracking	Applicants view their submitted applications and current status.
5	Administrator dashboard	Administrators view submitted applications, filter records and update statuses.
6	Status management	Workflow from submission to administrative review and final selection or rejection.
7	Dashboard statistics	Records grouped by department and status give an overview of admission activity.

C. Application Status Workflow

An application enters the system as submitted, is reviewed by an administrator, and ends in a final selection or rejection (Fig. 2). Status changes are restricted to administrator accounts.

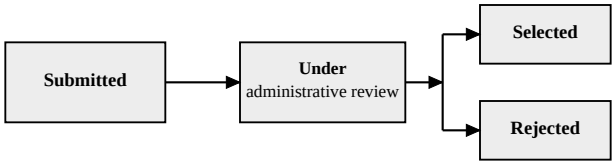

Fig. 2. Application status workflow supported by the portal.

V. VALIDATION AND ACCESS CONTROL

An important design decision was to enforce validation and access control on the backend instead of depending only on the frontend, since interface checks alone can be bypassed by sending requests directly to the API. Table III lists the measures. They help protect applicant information and support the security requirements defined in the project's SRS [1].

TABLE III
BACKEND VALIDATION AND ACCESS-CONTROL MEASURES

Measure	Implementation
Input validation	The backend checks submitted information before processing it.
Password security	Passwords are hashed with bcrypt rather than stored as plain text.
Authentication	JWT tokens authenticate users and protect restricted API endpoints.
Role-based authorisation	Applicant and administrator operations are separated; administrative functions require administrator authorisation.
Record protection	Applicants can access their own application records, not those of other applicants.
Document validation	Uploads are restricted to supported file types and a maximum size of 5 MB.

VI. LIMITATIONS AND FUTURE IMPROVEMENTS

The current implementation is a POC and does not represent a complete production-ready admission system. The following points describe its limits and the improvements planned for future versions.

1) Demonstration payment. Payment is currently a demonstration payment-status feature. A real payment gateway should be integrated.

2) Registration verification. Email or OTP verification should be introduced during applicant registration.

3) Document verification. Document verification needs additional administrative controls.

4) History and notifications. More detailed application history and notifications should be added.

5) Production security. Stronger production-level security, logging and error monitoring are required.

6) Testing. Complete unit, integration, performance and security testing has not yet been conducted.

7) Deployment. The application should be deployed on a production server with appropriate database and file-storage services.

VII. CONCLUSION

The MEAN stack POC demonstrates how a web-based admission portal can be built with MongoDB, Express.js, Angular and Node.js. It implements the essential applicant and administrator workflows: registration, application submission, document upload and status tracking. It also shows the importance of backend validation, authentication and role-based access control in protecting application data. Although several features need further development before production deployment, the POC establishes a working foundation for the IEM Admission Portal that can be extended to support the complete admission process.

REFERENCES

[1] Team 404 Brain Not Found, "Software Requirements Specification: IEM Admission Portal," course document, PCCCS594 – Software Engineering and Project Management Laboratory, Institute of Engineering and Management, Kolkata, India.

[2] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015.

[3] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf.*, 1999, pp. 81–91.

PROJECT LINKS AND TEAM

Live demo
https://iem-admission-404bn.web.app/
Scan the QR code to open the running application.

Source code (GitHub)
https://github.com/GhostMitra/SWE-Lab-C6-P10-248-220-227-243-232
Scan the QR code to open the repository.

TABLE IV
TEAM MEMBERS AND RESPONSIBILITIES

Name	Roll No.	Responsibility
Debarghaya Mitra	248	Project Manager
Arka Ghosh Dastider	220	UI/UX Designer
Jitesh Jana	243	Quality Assurance Engineer
Jatin Kumar	232	Software Architect
Diptomoy Das	227	Full-Stack Engineer